

https://social.dukla.ch/@andrzej

Andrzej Wąsowski

Advanced Programming

Probabilistic Programming with Probula

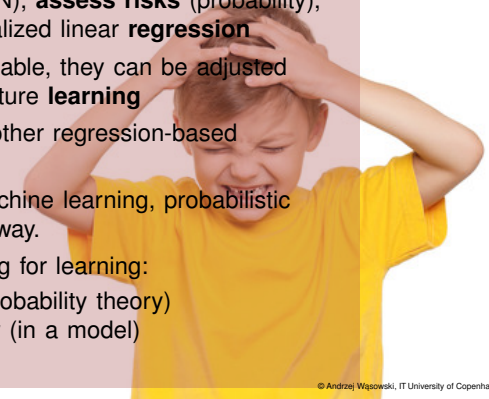
IT UNIVERSITY OF COPENHAGEN

SOFTWARE QUALITY RESEARCH

Probabilistic Programming

What and Why.

- A quickly growing field with applications in **data analytics and robotics**
- API/language to build a **probabilistic model** — so a probabilistic relation between some observed values and some conclusions/diagnosis
- You can use it to (among others): build **classifiers** (Y/N), **assess risks** (probability), predict values (expectation calculation), perform generalized linear **regression**
- Bayesian probabilistic models (most popular) are adaptable, they can be adjusted easily to new evidence showing up. This allows to capture **learning**
- So probabilistic programming is also an alternative to other regression-based machine learning methods.
- But unlike single-point-estimate models common in machine learning, probabilistic programming incorporates **uncertainty** in a principled way.
- Three nice things about using probabilistic programming for learning:
  - Based on sound and nice mathematical model (probability theory)
  - Gives a systematic way to capture what you know (in a model)
  - Allows learning the model parameters



Probabilistic Programming

A capstone project of Advanced Programming

- Probability distributions are **monads**, yay! (week 9)
- Probula probabilistic models are (reasonably) **referentially transparent**
- Random state is passed as a `given` argument (a **type class** instance)
- Probula is an **internal domain specific language** of Scala 3 (like our parser combinators, week 8), relying heavily on extension methods
- Models composed with `map`, `flatMap`, and `map2` (hello all ADPRO weeks!)
- Inference produces samples represented as `LazyLists` (week 4)
- The library is tested using dozens of **QuickCheck-style tests** (week 6)
- This lecture serves also as a teaser for the **Probabilistic Programming** course that runs the in Spring semester (by Raul Pardo and yours truly)

- **Probabilistic Programming** in Probula
  - Univariate `Dist[_]` and sampling
  - Multivariate models (detDep, probDep)
  - Bayesian linear regression (fluent API)
  - Fischer (for-yield)
- **Implementation** of Probula
  - Referential Transparency
  - Importance Sampling
  - The `Dist[_]` type constructor
  - Implementation of sampling
  - Units DSL
  - Fluent-API syntax
  - For-yield syntax
  - Testing

AGENDA

A 6-sided die

A univariate discrete random variable

```
1 val die: Dist[Int] = Uniform("die")(1, 2, 3, 4, 5, 6)
2 val roll: IData[Int] = die.sample(5.sampleSize)

4 val prime: Dist[Boolean] =              // Dist is a Functor
5   die.map { n => n == 2 || n == 3 || n == 5 }

7 val N = 10000.sampleSize
8 prime.sample(N).pr(true)                // Pr(prime) ~ 0.5

10 die.sample(N).mean                     // ~ 3.5
11 die.sample(N).median                   // 3 or 4
```

- `Dist[T]`: a distribution over values of type `T`
- `sample` draws an `IData[T]`, a lazy list of outcomes of type `T`
- `sample` requires a given `spire.random.rng.SecureJava`
- The dependence on spire is being replaced with dependence on `apache.commons.math3`
- `map` transforms a distribution; `Dist` is a **functor** (l.4–5)
- `pr`: probability of an event in a sample (l.8)
- `mean`, `median` give sample point estimates (l.10–11)
- `sampleSize`: opaque types and an injection function as an extension method provide a cheap way to implement units (l.2, l.7)

Two dice: A multivariate Model

Dependent variables, conditioning, variable hiding

```
1 def isPrime (n: Int) = n == 2 || n == 3 || n == 5
2 def isOdd   (n: Int) = n % 2 == 1

4 val primeGivenOdd = Probula
5   .uniform   ("die")   (1, 2, 3, 4, 5, 6)
6   .detDep    ("prime") { isPrime }
7   .detDep    ("odd")   { (die,prime) => isOdd (die) }
8   .condition { (die,prime,odd) => odd }  // only odd rolls
9   ._2                                    // only want to track prime

11 primeGivenOdd.sample(N).pr(true) // Pr(prime|odd) ~ 2/3
```

- **Multivariate** models conveniently represented by **tuples**
- Other data structures work; best automatic support for tuples
- `detDep` derives a **deterministic** variable, (l.6–7, tuples grow)
- `detDep` is just a `map` that adds to a tuple, instead of replacing it.
- The chain is a joint model over `(die, prime, odd)` (l.8)
- `condition` restricts to outcomes satisfying a predicate (l.8)
- Reading off probability of `prime` being true gives $\Pr(\text{prime} \mid \text{odd}) \approx 2/3$

Drawing without Replacement

Probabilistic dependencies; data types and lists as support

```
1 enum Marble:
2   case Red, Green
3 import Marble.*

5 val urn = List(Red, Red, Green)
6 val model = Probula // indices avoid duplication issues
7   .uniform (urn*)
8   .detDep  ("rest"){ fst => urn diff List(fst) }
9   .probDep ("2nd") { (fst, rest) => Uniform(rest*) }

11 model // Pr(2nd red | 1st green)
12   .matching { case (Green, _, _) => }
13   .sample(N)
14   .prMatching { case (_, _, Red) => }

16 model // Pr(2nd red | two colors left)
17   .matching { case (_, List(Red,Green), _) => }
18   .sample(N)
19   .prMatching { case (_, _, Red) => }
```

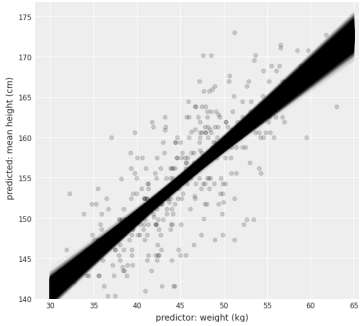
- `uniform` works over any finite sequence, including own types (l.7)
- A model variable can be **any** type; we are showing a `List` here, l.8 (types that support pattern matching and equality testing are easiest to use, besides numbers)
- `probDep` adds a dependent **random** variable whose distribution is a **function of earlier ones** (l.9);
- Probabilistically: `probDep` allows weaker correlation than `detDep`
- In the functional programming sense `probDep` is `flatMap`/bind that extends the model tuple
- `matching` conditions on a **pattern**; `prMatching` estimates probability of an event defined by a pattern (l.12,14,17,19)

Predicting Height from Weight

Bayesian linear regression

- α is height at mean weight,
- β defines the slope, height gain per kilogram
- σ the noise spread, uncertainty of the model given the data
- **priors** encode expectations
- Data sharpens the prior into a **posterior**, by **scoring** using a likelihood function
- We center the data to make the model interpretable
- DSL syntax: **fluent-style API**

$$\alpha \sim \mathcal{N}(178, 20)$$
$$\beta \sim \mathrm{LogN}(0, 1)$$
$$\sigma \sim \mathrm{Uniform}(0, 50)$$
$$\mu_i = \alpha + \beta\,(w_i - \overline{w})$$
$$h_i \sim \mathcal{N}(\mu_i, \sigma)$$

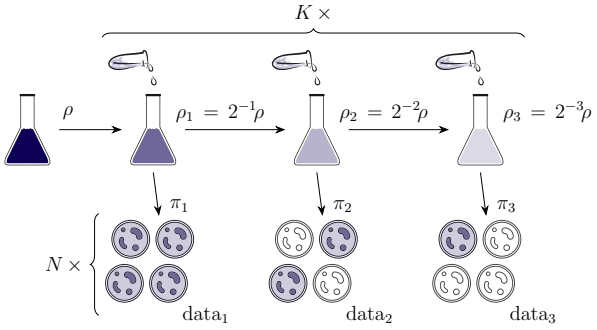



```
1 val model = Probula
2   .gaussian ("α")(178, 20)            // prior: intercept
3   .logNormal("β")(0, 1)               // prior: slope
4   .uniformC ("σ")(0, 50)              // prior: noise
5   .likelihood(data): (weight, height) =>
6     (α, β, σ) =>                       // predictor + likelihood
7       Gaussian(α + β*(weight - wbar), σ).observe(height)
```

Richard McElreath. *Statistical Rethinking: A Bayesian Course with Examples in R and Stan*. 2nd Ed. Chapman & Hall/CRC, 2020

Fisher's Dilution Assay

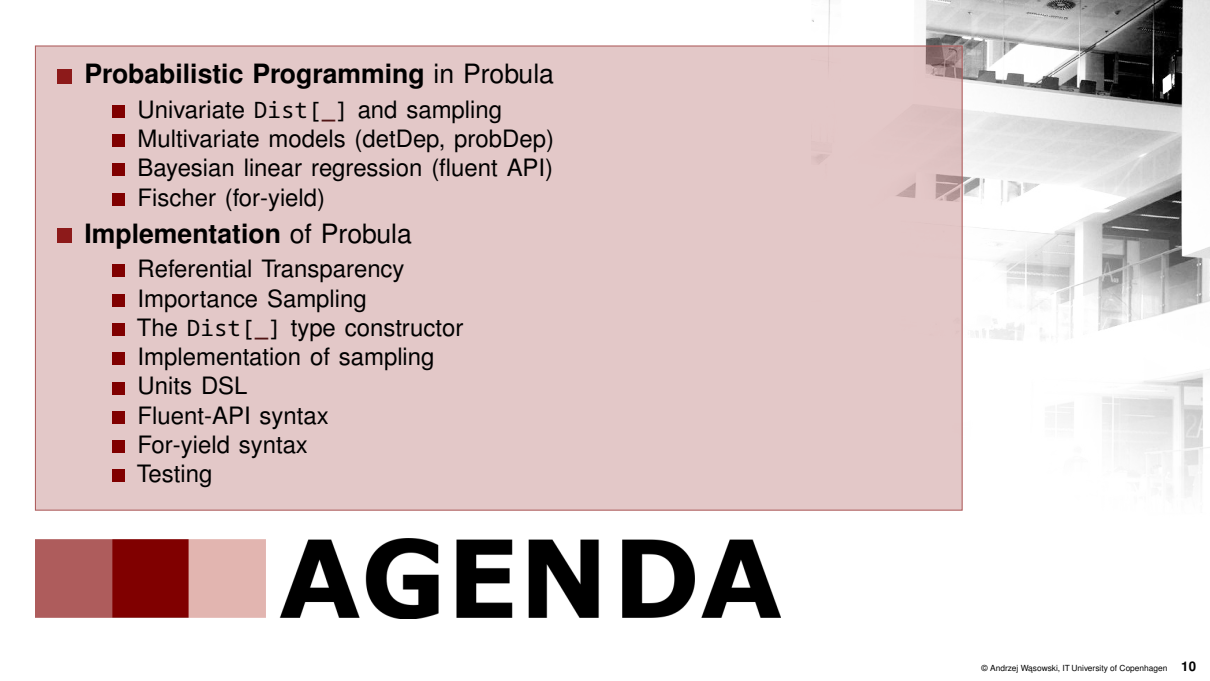
A more complex example of a Generalized Linear Model




```
1 for ρ <- UniformC(0, 2000)                          // prior
2     η = (1 to K).map { i => log(ρ) - i*log(2) }     // predictor
3     π = η.map { π => Cloglog.inverse (π) }          // link (cloglog)
4     _  <- π.plateD { πi => Binomial (N, πi) }.observing (data)
5 yield ρ                                             // ↑ likelihood
```

- Estimate the concentration ρ of organisms in a liquid
- Incubate `N` samples per stage;
- Each that grows holds ≥ 1 organism
- Dilute 1:1 over `K` stages (halve ρ each step) to eventually get both sterile and growing samples
- A `Binomial` count with a **cloglog** link gives the linear predictor
- Syntax based on for-yield, we do not name variables, but seems cleaner than fluent API

R. A. Fisher. On the mathematical foundations of theoretical statistics. *Phil. Trans. R. Soc. A*, 222(594–604):309–368, 1922



- **Probabilistic Programming** in Probula
  - Univariate `Dist[_]` and sampling
  - Multivariate models (detDep, probDep)
  - Bayesian linear regression (fluent API)
  - Fischer (for-yield)
- **Implementation** of Probula
  - Referential Transparency
  - Importance Sampling
  - The `Dist[_]` type constructor
  - Implementation of sampling
  - Units DSL
  - Fluent-API syntax
  - For-yield syntax
  - Testing

AGENDA

Referential transparency?

A design question; Should these three models be equivalent?

Model A

```
1 val x = Uniform(1, 2, 3, 4, 5, 6)
2 val model = x :* x
```

Model B

```
1 val x = Uniform(1, 2, 3, 4, 5, 6)
2 val y = x
3 val model = x :* y
```

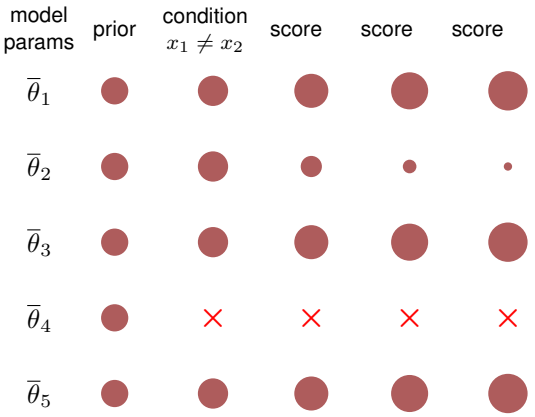
Model C

```
1 val x = Uniform(1, 2, 3, 4, 5, 6)
2 val y = Uniform(1, 2, 3, 4, 5, 6)
3 val model = x :* y
```

- Should aliasing change semantics? Should inlining? A question about referential transparency
- No. `:*` is an **independent product** — it samples each operand on its own, so A, B and C all agree (independent pairs, e.g. `(1,4) (3,3) (2,5) ...`)
- When we make all three **equivalent**: substitution and inlining are sound
- We need sthg else for correlation. This led to correlations only via `flatMap`/`map2`-based ops

Inference by Importance Sampling

`condition` drops samples; `score` reweights samples



- Each $\overline{\theta}_i$ is a matrix 6×6
- **Prior**: equal-weight samples of parameters of $X \times X$ (two dice)
- Only $\overline{\theta}_4$ that allows identical pairs (non-zero values on diagonal)
- $\overline{\theta}_2$ assumes no correlation, the others assume positive correlation ($x_1 \geq x_2$)
- `condition` **drops** impossible samples
- `score` **reweights** by data points; good fit begins to dominate; observed data ($x_2 > x_1$): (1,4) (2,6) (1,3) (3,6) (2,5);
- radius $\sim$ weight;
- `pr`/`mean` are weighted estimators

Modeling Distributions (`Dist[T]`)

```
1 trait Dist[T]:
2 self =>
3 def sample(using rng: RNG): IData[T] // abstract, must implement

5 // deterministic (Functor, derived)
6 def map[U](f: T => U): Dist[U] = new Dist[U]:
7   def sample(using rng: RNG) = self.sample.map(f)
8 def detDep[U](f: T => U): Dist[(T,U)] = self.map { t => (t,f(t))}

10 // random (Monad, derived)
11 def flatMap[U](f: T => Dist[U]): Dist[U] = new Dist[U]:
12   def sample(using rng: RNG) =
13     self.sample.flatMap { t => f(t).sample(1.sampleSize) }
14 def probDep[U](f: T => Dist[U]): Dist[(T,U)] =
15   self.flatMap { t => f(t).map { u => (t,u) } }

17 // independent product -- pointwise on the sample streams
18 def map2[U, V](that: Dist[U])(g: (T,U) => V): Dist[V] = new Dist[V]:
19   def sample(using rng: RNG) = self.sample.map2(that.sample)(g)
20 infix def :*[U](that: Dist[U]): Dist[(T,U)] = self.map2(that)((_,_))
```

- `sample` is the only **abstract** operation; other combinators are derived from it
- The classic `map`/`flatMap` **hide**/override the source variables
- Convenience: `detDep` (like map) and `probDep` (random, like flatMap) **keep** the source variable
- `map2` fuses variables
- `:*`/`zip` is the **independent product**

How to implement `sample`?

```
1 // A sample is a lazy stream of weighted particles:
2 opaque type Scored[+T] = (_value: T, _logScore: LogScore)
3 opaque type Chain [+T] = LazyList[Scored[T]]

5 class Uniform[T](val name: Name)(values: T*) extends Dist[T]:
6   def sample(using rng: RNG): IData[T] =
7     val gen = spire.random.Uniform(0, values.size - 1)
8     val list = gen.toLazyList(rng).map(values).map(unScored)
9     IData(name, list)

11 // re-score ONE particle: add a log-weight (= multiply the weight):
12 def reScored(adj: LogScore): Scored[T] = (value, logScore + adj)

14 // re-score the whole sample
15 def reScored(f: Scored[T] => LogScore): Chain[T] =
16   toLazyList.map { s => s.reScored(f(s)) }

18 // likelihood(data)(f) re-scores by the summed log-likelihood:
19 self.sample.chain.reScored:
20   s => data.map { d => f(d)(s._value) }.sum
```

- A sample is a lazy stream of **weighted particles**
- Prior draws have weight 1.0 (`unScored`)
- `reScored` adds a log-weight to a particle; in log space, adding is **multiplying**
- `score`, `condition`, `likelihood` delegate to `reScored`

Units DSL using Opaque Types + Extension Methods

```scala
opaque type SampleSize = Int

extension (n: Int)
  def sampleSize: SampleSize =
    require(n > 0, "Sample size must be positive")
    n

val N = 10000.sampleSize   // checked

// then sample only accepts SampleSize
def sample[U >: T](n: SampleSize)(using RNG): IData[U]
```

- `opaque type SampleSize = Int`: a distinct type at compile time, plain `Int` at run time
- `Int.sampleSize` is the **only** way in; it checks `n > 0` once, at construction
- Same recipe for `Prob` (`0.3.pr`) and `LogScore` (`1.5.lscore`)
- Easy to add your own for model variables

Implementing the fluent API

The `Dist` extensions and the `Probula` entry point (names elided)

```scala
object Probula:
  def uniform[T](values: T*): Dist[T] = Uniform(values*)
  def gaussian(mean: Double, stdDev: Double): Dist[Double] =...

extension [T] (self: Dist[T]) (using NotGiven[T <:< Tuple])

  def detDep[U](f: T => U): Dist[(T, U)] =
    self.map { t => (t, f(t)) }

  def probDep[U](f: T => Dist[U]): Dist[(T, U)] =
    self.flatMap { t => f(t).map { u => (t, u) } }

  infix def :* [U](that: Dist[U]): Dist[(T, U)] =
    self.map2(that) { identity }

  def gaussian(mean: Double, stdDev: Double): Dist[(T, Double)] =
    self :* Gaussian(mean, stdDev)
```

- `Probula`: thin forwarders to each distribution's constructor — the entry point for a chain
- `detDep`/`probDep`: same combinators as the `Dist` trait slide, but really `extensions` on `Dist[T]`, not trait members (why: next slide)
- `:*` (infix `zip`) tuples two distributions; the result is still a `Dist[(...)]`, so the same extensions apply again — chains keep growing
- `gaussian`, `bernoulli`, etc.: just sugar wrapping `:*` with a friendly name

Why so many `extensions`?

One Scala 3 mechanism, several different DSL jobs

- Beyond the unit-safety wrappers (`Int.sampleSize`, `Double.pr`, `Double.lscore`) and the fluent combinators just shown, Probula defines roughly **40 more `extension` blocks** across a dozen source files
- The fluent combinators **must** be extensions of `Dist` for two reasons:
  - Distinguish the version for tuples and not-tuples (`using NotGiven[T <:< Tuple]` constraint); Not resolved on trait methods in current Scala.
  - If any of the variants is an extension all of them need to be, as otherwise a trait method wins.

Enabling `for/yield`, and `Dist` as a monad

`flatMap+map` give you `for-yield`; `flatMap+unit` prove it's a monad

```
1 class Dirac[T](value: T) extends Dist[T]:
2   def sample(using rng: RNG): IData[T] =
3     IData(chain(LazyList.continually(unScored(value))))

5 given Monad[Dist] with
6   def unit[A](a: => A): Dist[A] = Dirac(a)

8   extension [A](fa: Dist[A])
9     def flatMap[B](f: A => Dist[B]): Dist[B] = fa.flatMap(f)
```

- `for/yield` is just sugar for `flatMap+map`, `Dist` has both
- with `Dirac(a)` providing `unit` we also get `Dist` to be a monad
- Probula does not instantiates the course's `Monad[F[_]]` typeclass for `Dist`, but the slide shows how this could be done
- `fa.flatMap(f)` resolves to `Dist`'s own method

N. Ramsey, A. Pfeffer. *Stochastic Lambda Calculus and Monads of Probability Distributions.* POPL 2002

Testing probabilistic code: QuickCheck style

ScalaCheck generates arbitrary inputs; sampling needs a statistical tolerance, not ==

```
1 property("Poisson.sample's mean converges to λ") =
2   val N = 100
3   forAll(Gen.chooseNum[Double](1.0, 50.0)): lambda =>
4     val p = Poisson("x")(lambda)
5     val actual = p.sample(N.sampleSize).mean
6     val ε = 5 * math.sqrt(lambda / N)
7     (math.abs(actual - lambda) <= ε) :|
8       s"expected mean≈$lambda, got $actual"
```

- `forAll` draws **arbitrary** rates λ
- For each λ it draws `N` real samples and checks the **empirical** mean
- ε is $5\times$ the real standard error of a Poisson mean, $\sqrt{\lambda/N}$

Koen Claessen and John Hughes. *QuickCheck: A Lightweight Tool for Random Testing of Haskell Programs.* ICFP 2000

Next Week

- Interpreters of programming languages as computation in monads
- Reading: Phil Wadler's paper
- See you next week ...